src/agent.py

Agentic RAG — a from-scratch Claude tool-use loop over the retriever

This is the methodology track's core. Unlike naive RAG, the agent decides how to retrieve: it decomposes a question, searches for each part, judges whether it has enough, and searches again — up to a cap — before answering. The loop is hand-written so every decision is a visible, logged message.

Walkthrough of all 182 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Agentic RAG — a from-scratch Claude tool-use loop over the retriever.
 2
 3  What makes this *agentic* rather than just RAG? Naive RAG (rag.py) retrieves once,
 4  up front, and hopes that single search grabbed everything. This agent instead
 5  *decides* how to retrieve: it can break a question into parts, search for each,
 6  read what came back, judge whether it has enough, and search again — before it
 7  answers. It runs a real Claude tool-use loop:
 8
 9      Claude, given a `search_corpus` tool, chooses a query
10        -> we run the retrieval and hand back the chunks
11        -> Claude reads them, decides: enough? if not, search again (new query)
12        -> ... up to MAX_AGENT_STEPS times ...
13        -> Claude writes the final grounded, cited answer
14
15  We implement the loop by hand — not with a framework — because explainability is
16  a hard requirement here: every decision the agent makes is a visible message we
17  log, so the notebook can show its reasoning trace. It uses the same VectorStore
18  as naive RAG (one definition of retrieval in the repo), so any quality difference
19  between the two is about the *loop*, not about a different search.
20
21  The pay-off this is built to demonstrate: on multi-hop questions — the kind that
22  need two different pieces of evidence — a single retrieval usually misses one of
23  them, and the agent's second, reformulated search is what recovers it.
24  """
```

Lines 1 to 24 are the module docstring. They define what makes this agentic (the model decides how to retrieve rather than retrieving once), sketch the tool-use loop, explain why it is built by hand rather than with a framework (explainability), and state the pay-off it demonstrates: on multi-hop questions a single retrieval misses one piece of evidence and the agent's second search recovers it.

```
25
26  from __future__ import annotations
27
28  from dataclasses import dataclass, field
29  from typing import Any
30
31  from . import config, llm
32  from .rag import extract_cited_ids
33  from .vectorstore import Retrieved, VectorStore
```

Lines 25 to 33 are imports: postponed annotations (line 26), dataclass and field for the result types (line 28), Any for message typing (line 29), config and the llm wrapper (line 31), extract_cited_ids reused from rag.py (line 32), and Retrieved plus VectorStore from the shared vector store (line 33) — the same retriever naive RAG uses.

```
34
35  # The agent's guidance. The key instructions: decompose multi-part questions and
36  # search for each part separately (this is what beats single-shot retrieval),
37  # only answer once you have the evidence, ground every claim, and cite arXiv ids.
38  SYSTEM_PROMPT = (
39      "You are a research assistant answering questions about machine-learning "
40      "papers. You have a `search_corpus` tool that returns passages from a corpus "
41      "of arXiv papers. Strategy: if a question has multiple parts or needs more "
42      "than one fact, search for each part separately rather than in one broad "
43      "query — a focused query retrieves better. Read what each search returns "
44      "before deciding whether you need another. When you have enough evidence, "
45      "write the answer using ONLY the retrieved passages; do not use outside "
46      "knowledge. Cite the arXiv ids you used in square brackets, e.g. [2401.01234]. "
47      "If the corpus cannot answer the question, say so plainly."
48  )
```

Lines 34 to 48 define SYSTEM_PROMPT, the agent's guidance. The comment (lines 35 to 37) highlights the key instruction; the string (lines 38 to 48) tells the model it has a search_corpus tool and to decompose multi-part questions into separate focused searches (which beats one broad query), to answer only from retrieved passages, to cite arXiv ids, and to say plainly when the corpus cannot answer.

```
49
50  # The one tool the agent may call. A tight schema — one string query — keeps the
51  # agent's choices legible and the loop easy to reason about.
52  SEARCH_TOOL = {
53      "name": "search_corpus",
54      "description": (
55          "Search the arXiv paper corpus for passages relevant to a query. "
56          "Returns the most semantically similar passages with their arXiv ids. "
57          "Call this multiple times with different focused queries to gather "
58          "evidence for different parts of a question."
59      ),
60      "input_schema": {
61          "type": "object",
62          "properties": {
63              "query": {
64                  "type": "string",
65                  "description": "A focused search query for one fact or sub-question.",
66              }
67          },
68          "required": ["query"],
69          "additionalProperties": False,
70      },
71  }
```

Lines 49 to 71 define SEARCH_TOOL, the single tool the agent may call, as the JSON schema Claude's tool-use API expects. It has a name (line 53), a description telling the model to call it multiple times with focused queries (lines 54 to 59), and an input_schema (lines 60 to 70) declaring one required string parameter, query, with additionalProperties false so the model cannot invent extra fields. A tight schema keeps the agent's choices legible.

```
 72
 73
 74  @dataclass
 75  class AgentStep:
 76      """One retrieval the agent chose to make — the auditable reasoning trace."""
 77
 78      query: str
 79      hits: list[Retrieved]
 80
```

Lines 72 to 80 define the AgentStep dataclass — one retrieval the agent chose to make. It holds the query it decided to run (line 78) and the hits that came back (line 79). The list of these steps is the auditable reasoning trace the notebook can display.

```
 81
 82  @dataclass
 83  class AgentAnswer:
 84      """The agent's final answer plus its full search trace.
 85
 86      `steps` is what makes the agent explainable: every query it decided to run and
 87      what came back, in order. `sources` is the de-duplicated union of everything
 88      retrieved across all steps — the evidence pool the answer was grounded in.
 89      """
 90
 91      question: str
 92      answer: str
 93      steps: list[AgentStep] = field(default_factory=list)
 94      sources: list[Retrieved] = field(default_factory=list)
 95      cited_ids: list[str] = field(default_factory=list)
 96      stopped_reason: str = "answered"  # "answered" or "max_steps"
 97
```

Lines 81 to 97 define the AgentAnswer dataclass — the final result plus the full trace. Fields: the question and answer (lines 91 to 92), the ordered list of steps (line 93), the de-duplicated union of all retrieved sources (line 94), the cited ids (line 95), and stopped_reason (line 96, either answered or max_steps). The three list fields use default_factory=list so each instance gets its own list.

```
 98
 99  def _format_tool_result(hits: list[Retrieved]) -> str:
100      """Render one search's hits into the text handed back to the agent."""
101      if not hits:
102          return "No passages found for that query."
103      return "\n\n".join(f"[{h.arxiv_id}] {h.title}\n{h.text}" for h in hits)
104
```

Lines 98 to 104 define _format_tool_result, which renders one search's hits into the text handed back to the model. Lines 101 to 102 return a clear no-passages message when a search finds nothing; line 103 otherwise joins each hit as its bracketed id, title, and text — the same citable format the prompt uses.

```
105
106 def answer(store: VectorStore, question: str, k: int = config.TOP_K) -> AgentAnswer:
107     """Run the agentic loop and return the answer plus its search trace.
108
109     This is a manual Claude tool-use loop: we call the model, and while it keeps
110     asking to use the search tool we run the search and feed the results back,
111     appending every turn to the running message history. The loop is bounded by
112     MAX_AGENT_STEPS so a confused agent can't retrieve forever — after the cap we
113     ask it once more, tools disabled, to answer with what it has.
114     """
115     messages: list[dict[str, Any]] = [{"role": "user", "content": question}]
```

Lines 105 to 115 begin answer(), the agentic loop. The docstring (lines 107 to 114) describes it as a manual tool-use loop bounded by MAX_AGENT_STEPS with a forced-answer turn at the cap. Line 115 seeds the message history with the user's question.

```
116     steps: list[AgentStep] = []
117     all_hits: dict[str, Retrieved] = {}   # de-dupe the evidence pool by chunk_id
118
```

Lines 116 to 118 initialise two accumulators: steps (the trace, line 116) and all_hits, a dict keyed by chunk_id (line 117) that de-duplicates the evidence pool so the same chunk retrieved by two different searches is stored once.

```
119     for _ in range(config.MAX_AGENT_STEPS):
120         # Inside the loop the search tool is always offered; the model itself
121         # decides whether to search again or answer. The step cap below is the
122         # backstop if it never chooses to stop searching.
123         result = llm.call(messages=messages, system=SYSTEM_PROMPT, tools=[SEARCH_TOOL])
```

Lines 119 to 123 open the bounded loop (up to MAX_AGENT_STEPS iterations). The comment (lines 120 to 122) notes the search tool is always offered inside the loop and the model itself decides whether to search or answer. Line 123 calls Claude with the messages, system prompt, and the search tool.

```
124
125         # Record the assistant turn verbatim so the tool-use blocks are preserved
126         # for the next request (the API needs them to match the tool results).
127         messages.append({"role": "assistant", "content": result["content"]})
```

Lines 124 to 127 append the assistant's turn to the history verbatim. The comment (lines 125 to 126) explains why this matters: the API needs the original tool-use blocks preserved so the tool results sent next can be matched to them.

```
128
129         tool_uses = [b for b in result["content"] if b["type"] == "tool_use"]
```

Lines 128 to 129 extract any tool_use blocks from the model's response — the searches it is requesting this turn.

```
130         if not tool_uses:
131             # No tool call -> the agent chose to answer. We're done.
132             text = llm.first_text(result)
133             sources = list(all_hits.values())
134             return AgentAnswer(
135                 question=question,
136                 answer=text,
137                 steps=steps,
138                 sources=sources,
139                 cited_ids=extract_cited_ids(text, sources),
140                 stopped_reason="answered",
141             )
```

Lines 130 to 141 handle the answer case: if there were no tool calls (line 130) the agent chose to answer, so it pulls the text (line 132), snapshots the de-duped sources (line 133), and returns an AgentAnswer with cited_ids computed from the retrieved sources and stopped_reason set to answered (lines 134 to 141).

```
142
143         # Run every requested search and return all results in one user message
144         # (the API expects one tool_result per tool_use, batched together).
145         tool_results = []
146         for tu in tool_uses:
147             query = tu["input"]["query"]
148             hits = store.search(query, k=k)
149             steps.append(AgentStep(query=query, hits=hits))
150             for h in hits:
151                 all_hits[h.chunk_id] = h
152             tool_results.append(
153                 {
154                     "type": "tool_result",
155                     "tool_use_id": tu["id"],
156                     "content": _format_tool_result(hits),
157                 }
158             )
159         messages.append({"role": "user", "content": tool_results})
```

Lines 142 to 159 handle the search case: for each requested search (line 146) it reads the query (line 147), runs the shared retriever (line 148), records the step (line 149), folds the hits into the de-duped pool (lines 150 to 151), and builds a tool_result carrying the matching tool_use_id and the formatted hits (lines 152 to 158). Line 159 appends all results as one user message — the API expects one tool_result per tool_use, batched together.

```
160
161     # Hit the step cap without answering: ask once more with tools off so the
162     # model is forced to commit to an answer using the evidence gathered so far.
163     messages.append(
164         {
165             "role": "user",
166             "content": (
167                 "You have gathered enough evidence. Now write the final answer "
168                 "using only the passages retrieved so far, citing arXiv ids."
169             ),
170         }
171     )
172     result = llm.call(messages=messages, system=SYSTEM_PROMPT, tools=None)
173     text = llm.first_text(result)
174     sources = list(all_hits.values())
175     return AgentAnswer(
176         question=question,
177         answer=text,
178         steps=steps,
179         sources=sources,
180         cited_ids=extract_cited_ids(text, sources),
181         stopped_reason="max_steps",
182     )
```

Lines 160 to 182 are the step-cap backstop, reached only if the model never chose to answer within the loop. Lines 163 to 171 append a final user message forcing an answer from the evidence gathered. Line 172 calls Claude once more with tools set to None (so it must answer, not search). Lines 173 to 181 return an AgentAnswer just like the normal path, but with stopped_reason set to max_steps so callers can see the cap was hit rather than a natural stop.